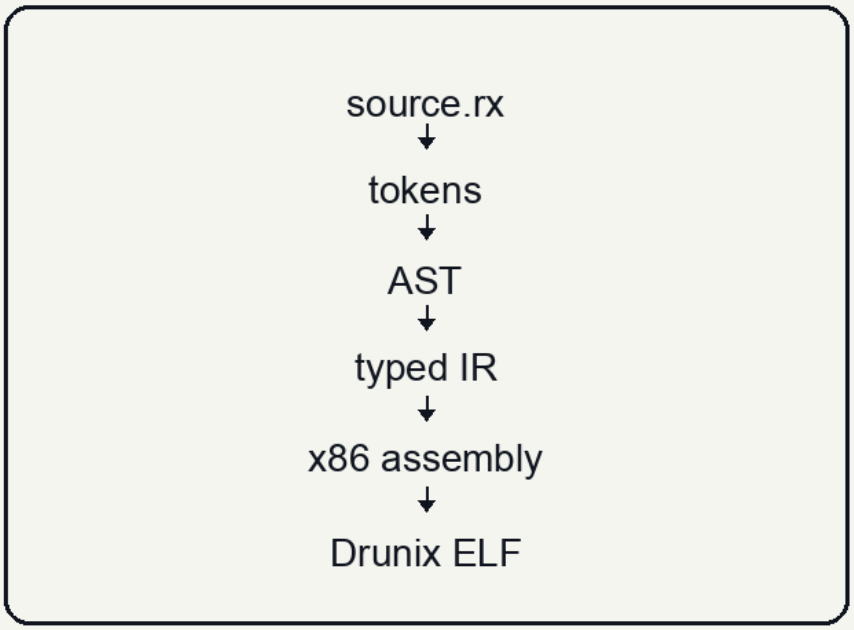
REXC

Building a Systems Language Compiler for Drunix



My journey in writing a compiler for my OS

A companion to the Drunix book

Drew Tennenbaum

2026

Contents

Part I - From Source Text to Syntax Tree 1
Chapter 1 - Source Text and Tokens 2
Chapter 2 - Parsing Functions and Statements 4
Part II - Giving the Program Meaning 5
Chapter 3 - Semantic Analysis and Types 6
Chapter 4 - Lowering to Typed IR 8
Part III - From Typed IR to Assembly 9
Chapter 5 - Emitting x86 Assembly 10
Part IV - Linking Rexc Programs into Drunix 12
Chapter 6 - Building a Drunix ELF Program 13

Front Matter

Part I - From Source Text to Syntax Tree

When Rexc begins, there is no program in the compiler's hands yet. There is only text: bytes from a `.rx` file, arranged by a human into names, keywords, punctuation, and literals. The first job is not to decide whether the program is correct. The first job is to turn that text into a shape the rest of the compiler can reason about.

Part I follows that transformation. Chapter 1 starts with the raw source file and explains how Rexc breaks it into tokens. A token is a classified piece of source text, such as `fn`, `main`, `i32`, `{`, or `return`. Chapter 2 takes those tokens and builds an **AST** (Abstract Syntax Tree, the tree-shaped representation of the program's grammar). Once the AST exists, the compiler no longer has to remember where every character appeared. It can talk about functions, statements, expressions, and types.

By the end of Part I, Rexc has not proved the program is meaningful yet. It has proved something more basic: the source has the grammar of a Rexc program. That is enough to hand the tree to the semantic analyzer, which is where the compiler starts asking what the program means.

Chapter 1 - Source Text and Tokens

The Moment the Compiler Opens the File

Rexc begins in a quieter place than Drunix. There is no firmware handoff, no CPU mode to claim, and no interrupt table waiting to be installed. The first state is simply a source file loaded into memory. The command-line driver has read the `.rx` file into a `SourceFile`, and from this point forward every diagnostic can point back to a path, line, and column.

That location tracking matters immediately. A compiler spends a lot of its time saying no, and a useful no has to name the exact part of the program that made the compiler stop. Before Rexc knows whether a function exists or a type is valid, it must know where every piece of text came from.

Turning Characters into Tokens

The first compiler stage is the **lexer**. A lexer is the part of a compiler that reads raw characters and groups them into tokens. A token is a small classified unit of source text: an identifier, a keyword, a number, a string literal, or punctuation such as `(` and `{`.

Consider this small Rexc program:

```
fn main() -> i32 {
    return 42;
}
```

The lexer does not see a function yet. It sees a sequence of characters. Its job is to walk from left to right and produce a stream with more structure:

Source text	Token role
`fn`	function keyword
`main`	identifier
`(`	left parenthesis
`)`	right parenthesis
`->`	return-type arrow
`i32`	identifier used as a type name
`{`	left brace
`return`	return keyword
`42`	integer literal
`;`	statement terminator
`}`	right brace

This might look like a small improvement, but it is the first important boundary in the compiler. Later stages do not need to ask whether the letter `f` followed by the letter `n` is a keyword. They receive a token that already answers that question.

Whitespace, Comments, and Source Locations

Whitespace separates tokens, but it is not itself part of the program shape. The lexer skips spaces, tabs, and newlines after using them indirectly through the source-location table. Line comments are treated the same way: once the lexer sees `//`, it advances to the end of the line and resumes from the next character.

The important thing is that skipping text does not mean losing accountability. Every token keeps the offset where it began. When a later stage reports an unknown type or a mismatched operand, it can ask the original `SourceFile` to translate that offset into a human-readable line and column. The compiler's memory of the source remains precise even after the source has been transformed into tokens.

Literals Keep Their Original Text

Integer literals carry two pieces of information. They carry the parsed numeric value where that is convenient, but they also keep the original decimal text. That second detail is easy to miss, and it matters.

Some Rexc integers are larger than the host type the parser uses for quick conversion. A `u64` literal such as `18446744073709551615` is a perfectly valid source value, but it does not fit into a signed 64-bit parser result. If the compiler threw away the original text too early, it could accidentally wrap the value and type-check the wrong number. Rexc avoids that by letting semantic analysis inspect the literal's decimal spelling later, when the expected type is known.

Strings and characters take a different path. The lexer decodes escape sequences such as newline and tab into the literal value the program meant. That gives the parser a clean token while still letting diagnostics point back to the source if an escape sequence is malformed or a literal is unterminated.

Where the Compiler Is by the End of Chapter 1

Rexc has now crossed its first boundary. The input is no longer anonymous text. It is a stream of tokens, and every token knows where it came from.

The compiler can recognise keywords, identifiers, punctuation, operators, integer literals, booleans, characters, strings, comments, and whitespace. It can report lexical errors with source locations. It has not yet decided whether the token stream forms a valid program. That is the parser's job, and it begins with the question the lexer deliberately avoided: what larger structure do these tokens describe?

Chapter 2 - Parsing Functions and Statements

From a Token Stream to a Program Shape

At the end of Chapter 1, Rexc had a token stream. That stream was cleaner than raw text, but it was still flat. The compiler knew that `fn` was a keyword and that `main` was an identifier, but it did not yet know that together they began a function definition.

The **parser** performs that next step. A parser is the compiler stage that checks whether tokens follow the language grammar and builds a structured representation of that grammar. Rexc uses ANTLR to generate its lexer and parser from `grammar/Rexc.g4`. That keeps the accepted syntax in one grammar file, while `src/parse.cpp` stays focused on invoking ANTLR and converting the generated parse tree into Rexc's own AST.

Items, Blocks, and Statements

Rexc source is organised around functions. At the top level, the parser expects either a function definition or an extern declaration. An extern declaration names a function that exists outside the current Rexc source file, which lets Rexc-generated code call into a runtime or library supplied by the final link.

Inside a function body, the parser builds statements. A statement is a piece of program structure that does something in sequence. Rexc currently has nine statement shapes:

Statement	What it means
`let`	create an immutable local value with an explicit type
`let mut`	create a mutable local value with an explicit type
assignment	update an existing mutable local
`return`	produce the function result
`if`	conditionally run one block
`if/else`	choose between two blocks
`while`	repeat a block while a condition is true
`break`	leave the nearest loop
`continue`	jump to the next check of the nearest loop

When the parser sees braces, it enters a block. A block is a sequence of statements surrounded by `{` and `}`. The parser does not yet decide whether a name is visible in that block or whether a return type is correct. It only preserves the shape so semantic analysis can answer those questions with the whole function in view.

Expressions and Precedence

Expressions are the parts of the language that produce values. A literal is an expression. A name is an expression. A function call is an expression. Binary operators such as + and < are expressions too, but they introduce a small problem: order.

In `1 + 2 * 3`, multiplication binds more tightly than addition. The parser must produce a tree where `2 * 3` is grouped first, then added to `1`. That rule is called **precedence**. Precedence is the parser's way of turning a flat operator sequence into the tree the programmer meant.

Rexc's current expression layers are:

Layer	Operators or forms
Logical or	`||`
Logical and	`&&`
Comparison	`==`, `!=`, `<`, `<=`, `>`, `>=`
Additive	`+`, `-`
Multiplicative	`*`, `/`
Unary	unary `-`, `!`
Primary	literals, names, calls, parenthesised expressions

These grammar layers give each operator family a clear place. A comparison can contain arithmetic on either side. Arithmetic can contain calls and names. A parenthesised expression can override the default grouping when the source needs a different shape.

The AST as the Parser's Handoff

The parser's output is an **AST** (Abstract Syntax Tree). "Abstract" means the tree keeps the program structure but leaves behind details that no longer matter. Whitespace is gone. Comments are gone. Parentheses that only guided grouping are gone. What remains is the shape the compiler will analyze: functions contain parameters and statements, statements contain expressions, and expressions contain other expressions.

This handoff is intentionally not typed. A binary expression records its operator and operands, but it does not yet know whether `+` is being applied to integers or accidentally to booleans. A type name records the spelling from the source, but it is not yet resolved into the compiler's primitive type model. That restraint keeps the parser simple. It answers the grammar question and then stops.

Where the Compiler Is by the End of Chapter 2

Rexc can now turn valid source text into a syntax tree. It understands function definitions, extern declarations, typed parameters, immutable and mutable typed locals, assignments, returns, conditionals, while loops, calls, literals, unary expressions, binary arithmetic, comparisons, boolean operators, `break`, and `continue`.

The compiler still has not proved the program is meaningful. The tree might refer to an unknown function. A return statement might produce the wrong type. An assignment might target an immutable local. An `if` or `while` condition might be an integer instead of a boolean. A `break` might appear outside a loop. Those are semantic questions, and the next stage is where Rexc begins to answer them.

Part II - Giving the Program Meaning

The parser gives Rexc a tree, but a tree is only grammar. A grammatically valid program can still be wrong. It can return a string from a function that promised an integer. It can use a local before the local exists. It can compare values whose types do not match. It can assign to a local that was never declared mutable. It can try to `break` when there is no loop to break out of.

Part II covers the point where Rexc starts proving facts about the program. Chapter 3 performs semantic analysis: it resolves names, checks primitive types, validates calls, and rejects programs that do not make sense. Chapter 4 then lowers the checked AST into a typed intermediate representation, or IR. The IR is smaller than the AST and more convenient for code generation, but it keeps the facts semantic analysis proved.

By the end of Part II, Rexc has moved from "this source has the right grammar" to "this program has a coherent typed meaning." That is the contract the backend needs before it can safely emit assembly.

Chapter 3 - Semantic Analysis and Types

When Grammar Is Not Enough

By the time semantic analysis begins, the parser has already done its job. Rexc has a tree made of functions, statements, expressions, and type names. That tree is syntactically valid. It is not necessarily meaningful.

Semantic analysis is the compiler stage that checks meaning. It asks whether names refer to something real, whether types line up, whether calls have the right number of arguments, and whether literals fit in the type the program asked for. This is the first stage that can say, "the program is written in the language's grammar, but it still cannot run."

The Primitive Type Model

Rexc currently has a small systems-language type set:

Family	Types
Signed integers	`i8`, `i16`, `i32`, `i64`
Unsigned integers	`u8`, `u16`, `u32`, `u64`
Other scalar values	`bool`, `char`, `str`

A type name in the AST is only source spelling. During semantic analysis, Rexc parses that spelling into a primitive type value. That value is what the rest of the compiler compares. Two source locations that both say `i32` become the same internal type. A misspelled type never gets that far; it becomes a diagnostic attached to the source location where the bad name appeared.

The distinction between signed and unsigned integers matters beyond range checking. It affects division and comparisons later in code generation. Signed values use signed CPU instructions and signed condition codes. Unsigned values use unsigned ones. Semantic analysis records enough type information for the backend to choose correctly without reinterpreting source text.

Names, Locals, and Local Mutation

The analyzer walks each function with a table of visible local names. Function parameters enter the table first. They are visible in the function body, but they are not assignable. A `let` statement checks its initializer before inserting the new name, so a declaration such as `let x: i32 = x;` is still an unknown-name error rather than a self-reference that silently works.

Rexc separates declaration from mutation. A plain `let` creates an immutable local. A `let mut` creates a mutable local. Assignment is only valid when the target name already exists and was declared mutable:

```
fn count() -> i32 {
    let mut value: i32 = 0;
    value = value + 1;
    return value;
}
```

The right side of an assignment is checked against the target local's declared type. Assigning a `bool` into an `i32`, assigning to a plain `let`, assigning to a parameter, or assigning to an unknown name all produce diagnostics.

Block Scopes

Branches get their own copies of the local table. When Rexc analyzes an `if` statement, it checks the condition, then analyzes the `then` body with one local table and the `else` body with another. A local declared inside a branch is visible later in that same branch, but it does not leak out to the code after the `if`. That rule keeps the current

language simple and prevents a later statement from depending on a local that might not have been declared at runtime.

Loops use the same scoping idea. A `while` condition is checked in the surrounding local table, and the loop body is analyzed with a copy of that table. Locals declared inside the loop body are visible later in that body, but they do not leak out after the loop. Mutations to already-visible mutable locals are allowed because assignment changes an existing slot rather than introducing a new name.

Loop-control statements add one more fact for sema to track: whether the current statement is inside a loop. A `break` exits the innermost loop, and a `continue` skips to that loop's next condition check. Both are meaningful only while a loop is being analyzed. If either appears in a function body or an `if` branch that is not nested inside a loop, Rexc reports a diagnostic.

Expressions Must Produce the Promised Type

Expected types flow into expressions from their context. A return expression is checked against the function's return type. A local initializer is checked against the local's declared type. A function-call argument is checked against the corresponding parameter type.

This is especially important for integer literals. The literal text `1` does not name a width on its own. If it initializes a `u8`, Rexc checks it as a `u8`. If it returns from an `i64` function, Rexc checks it as an `i64`. The literal carries its decimal spelling from the lexer so this range check can be done without accidental host-side overflow.

Arithmetic operators require integer operands of the same type and produce that same type. Comparison operators also require same-typed integer operands, but they produce `bool`. An `if` or `while` condition must already be `bool`; Rexc does not silently treat integers as truthy or falsy.

Boolean operators live fully in the `bool` world. Unary `!` requires a `bool` operand and produces `bool`. The `&&` and `||` operators also require both operands to be `bool` and produce `bool`. Semantic analysis checks both sides even though the backend later emits short-circuit jumps, because both operands must be valid Rexc expressions before runtime control flow matters.

Where the Compiler Is by the End of Chapter 3

Rexc can now reject programs that are grammatically valid but semantically wrong. It knows which functions exist, which locals are visible, which primitive types are valid, which calls match their signatures, and which expressions produce which types. It also knows which locals are mutable, which assignments are legal, which logical expressions are boolean, which branch and loop conditions are boolean, and which `break` and `continue` statements are actually inside loops.

The compiler has not emitted anything yet. It still holds the source-shaped AST, now checked for meaning. The next step is to lower that tree into a smaller typed representation built for the backend rather than for the parser.

Chapter 4 - Lowering to Typed IR

Why the Compiler Needs Another Shape

After semantic analysis, Rexc has a checked AST. That AST is useful because it looks like the source program. It is less ideal for code generation, because it still carries parser-facing details: source type names, source statement classes, and expression nodes arranged for grammar first.

Rexc lowers the AST into **IR** (Intermediate Representation, a compiler-owned program shape between the frontend and backend). The IR is not source text and it is not assembly. It is the compiler's working form: typed, smaller than the AST, and direct enough for assembly emission.

What the IR Keeps

The IR keeps the facts the backend needs. Functions have names, parameters, return types, and statement bodies. Values carry resolved primitive types. Integer literals keep their decimal text. Calls carry the callee name and the lowered argument values. Assignments carry a target local name and a typed value to store. Branches and loops carry typed conditions and lowered statement bodies.

The main shift is that source type names disappear. Once semantic analysis has proved that `i32` is a valid primitive type, later stages do not need to parse the string `i32` again. They receive a primitive type value directly. That means code generation can ask simple questions: is this type signed? is it 64-bit? is this target able to emit it?

Expected Types Still Matter During Lowering

Lowering mirrors part of semantic analysis because literals need context. If a function returns `u64` and the source says `return 1;`, the lowered integer value should be a `u64` integer value, not a default `i32`. If a call passes `1` into an `i8` parameter, the lowered argument should remember that it is `i8`.

This does not mean lowering is re-checking the whole program. Semantic analysis has already decided whether the program is valid. Lowering follows the same expected-type paths so the IR preserves the resolved type decisions that sema already made.

Control Flow Becomes Backend-Friendly

An `if/else` node in the AST is still a source-level statement. In the IR it becomes a branch statement with three direct pieces: the condition value, the then body, and the else body. The condition's type is `bool`, not just "whatever expression the source wrote." That is the contract code generation relies on when it later emits a compare-against-zero and a conditional jump.

`while` lowers in the same spirit. The IR loop node carries a boolean condition value and a lowered body. The backend does not need to rediscover that the condition is legal or that the body is scoped; semantic analysis already proved those facts, and lowering preserved them.

`break` and `continue` lower to marker statements. They do not carry their target labels in IR because labels are a backend detail. Semantic analysis has already proved they are inside a loop, so code generation can resolve them to the innermost loop it is currently emitting.

The IR also keeps branch-local statements contained in their branch bodies. Loop-local statements stay contained in loop bodies for the same reason. That matches semantic analysis and makes stack-slot assignment possible later: each `let` statement can be assigned its own storage even if another branch or loop body uses the same source name.

Mutation Targets Existing Storage

The IR does not need a separate "mutable local" flag. Mutability is a source language rule, and semantic analysis has already enforced it. By the time lowering creates an assignment statement, the target is known to be a visible mutable local and the assigned value is known to have the target's type.

That leaves the backend with a small instruction-level job: evaluate the right side and store the accumulator back into the existing stack slot for that local. Declarations allocate slots; assignments reuse them.

Where the Compiler Is by the End of Chapter 4

Rexc now holds a typed IR module. The source has been parsed, checked, and lowered into a backend-facing representation. Every value has a primitive type. Every branch and loop condition is already known to be boolean. Every assignment targets an existing local with a value of the right type. Every `break` and `continue` statement is known to be inside a loop. Every function body is a sequence of typed IR statements.

The compiler is ready to leave source-language territory. The next stage must take this typed IR and make it concrete for a CPU: stack frames, registers, instructions, labels, calls, and returns.

Part III - From Typed IR to Assembly

Once Rexc has typed IR, it has done the frontend's work. The remaining question is physical: how should this program run on an x86 CPU?

Part III follows the backend. Chapter 5 explains how Rexc maps typed values to stack slots and registers, emits arithmetic and comparisons, lowers calls into the target calling convention, stores assignments back into local slots, and turns `if/else` branches, `while` loops, `break`, and `continue` into labels and jumps. The same IR can target i386 or x86_64 because the backend owns the target-specific choices.

By the end of Part III, Rexc can write GNU assembler syntax. That assembly is not the final executable yet, but it is no longer an abstract program. It is a linear instruction stream that an assembler can turn into an ELF object file.

Chapter 5 - Emitting x86 Assembly

The Backend Receives a Typed Program

Code generation begins with an important advantage: it does not have to guess. The backend receives typed IR. It knows whether a value is signed or unsigned, whether a comparison produces `bool`, whether a local is a string pointer or an integer-sized scalar, whether an assignment targets existing local storage, and whether the selected target is i386 or x86_64.

The backend's job is to turn those facts into assembly. Assembly is a textual form of machine instructions. Rexc emits GNU assembler syntax, which means the next tool in the chain can be `as` or `x86_64-elf-as`.

Stack Frames and Local Slots

Every non-extern function gets a stack frame. A stack frame is the block of stack memory a function uses for its saved frame pointer, parameters, locals, and temporary call state. Rexc keeps the current model deliberately simple: locals live in fixed stack slots, and expression results flow through the accumulator register.

A `let` statement reserves one of those local slots and stores the initializer there. An assignment does not reserve a new slot. Rexc emits the right side into the accumulator, then writes that value back to the slot already associated with the target local. That distinction is why semantic analysis had to reject assignments to immutable locals before code generation began.

On i386, Rexc uses 32-bit slots for the currently supported scalar values. On x86_64, Rexc uses 64-bit slots and aligns the local frame to preserve the calling convention's stack expectations. Parameters follow the target ABI. An **ABI** (Application Binary Interface) is the binary contract that says where arguments go, which registers matter, how the stack is aligned, and how a function returns.

The two current targets differ most visibly at function calls:

Target	First argument path	Extra argument path
`i386`	stack	stack
`x86_64`	registers such as `%rdi` and `%rsi`	stack after register arguments

That difference is why the IR does not try to encode call mechanics. Calls are a backend decision.

Arithmetic, Division, and Comparisons

Simple arithmetic follows a repeated pattern. Rexc emits the left operand into the accumulator, saves it, emits the right operand, moves the right operand into a scratch register, restores the left operand, and then emits the target instruction. The instruction suffix changes by target: i386 uses forms such as `addl`, while x86_64 uses forms such as `addq`.

Division is more careful. x86 division instructions use an implicit dividend spread across the accumulator and remainder registers. Signed division requires sign extension first. Unsigned division requires clearing the remainder register first. Because semantic analysis preserved signedness, the backend can choose the right instruction shape without inspecting source text.

Comparisons produce `bool`. Rexc emits a compare instruction, then a condition code instruction that writes one byte: `1` if the condition is true, `0` if it is false. The result is then zero-extended back into the accumulator-sized register. Signed comparisons use signed condition codes; unsigned comparisons use unsigned ones.

Boolean Operators and Short-Circuit Jumps

Unary `!` is a small boolean transformation: emit the operand, compare the low byte against zero, set `%al` when the value was false, and zero-extend the result back into the full accumulator register.

The binary boolean operators are more interesting because `&&` and `||` short-circuit. For `&&`, Rexc emits the left operand and jumps directly to the false label if it is zero. Only a true left operand lets execution reach the right operand. For `||`, the shape is reversed: a true left operand jumps directly to the true label. In both cases, the final labels materialize a normal `0` or `1` result in the accumulator.

That means a Rexc expression such as `ready && expensive()` preserves the usual runtime promise: `expensive` is called only when `ready` is true.

Control Flow Becomes Labels and Jumps

An `if/else` statement arrives in IR as a boolean condition plus two statement bodies. Assembly has no tree-shaped branch node, so the backend turns that shape into labels and jumps.

First, Rexc emits the condition. Since conditions are `bool`, the accumulator holds zero or one. The backend compares the low byte against zero. If it is zero, execution jumps to the else label. If it is non-zero, execution falls through into the then body. After the then body, Rexc emits an unconditional jump over the else body to the end label.

That gives the CPU a concrete path through what used to be a tree:

Runtime condition	Instruction path
condition is false	jump to else label
condition is true	run then body, then jump to end label

The interesting part is that this is the same idea on both targets. The exact register names and instruction suffixes differ, but the control-flow shape is shared.

A `while` loop uses the same pieces with a different label layout. Rexc emits a start label, emits the loop condition, and jumps to the end label when the condition is false. If the condition is true, execution falls through into the body. After the body, an unconditional jump returns to the start label so the condition can be tested again.

While emitting a loop body, the backend remembers that loop's start and end labels. A `continue` statement becomes an unconditional jump to the current start label. A `break` statement becomes an unconditional jump to the current end label. For nested loops, the most recently entered loop is at the top of that label stack, so `break` and `continue` naturally target the innermost loop.

That creates the usual loop shape:

Runtime condition	Instruction path
condition is false	jump to loop end
condition is true	run body, then jump back to loop start

Where the Compiler Is by the End of Chapter 5

Rexc can now emit assembly for typed functions, locals, returns, calls, arithmetic, division, comparisons, boolean operators, strings, assignment, `if/else` branches, and `while` loops with `break` and `continue`. The i386 target is the default path for the current Drunix user runtime. The x86_64 target emits 64-bit Linux-compatible assembly using the System V calling convention.

The compiler still has not produced an executable by itself. Assembly is the input to the assembler, and the assembler produces an object file. To become a Drunix program, that object must be linked with the user runtime and the right linker script.

Part IV - Linking Rexc Programs into Drunix

Assembly is close to the machine, but it is not yet a runnable program. The assembler must turn it into an object file, and the linker must combine that object with the startup code and runtime library that Drunix expects every user program to have.

Part IV follows that final handoff. Chapter 6 explains how Rexc output fits into the Drunix userland build path: source becomes assembly, assembly becomes an ELF object, and the object links with `crt0.o`, `libc.a`, and the user linker script to produce a 32-bit Drunix ELF executable.

By the end of Part IV, the compiler's output has crossed the boundary from language artifact to operating-system artifact. Drunix does not need to know that Rexc source produced the binary. It only needs a valid executable with the right entry point and runtime contract.

Chapter 6 - Building a Drunix ELF Program

The Compiler Stops at Assembly

Rexc currently stops after assembly output. That is a deliberate boundary. The compiler owns source analysis, typing, lowering, and target assembly emission. The assembler and linker own the binary container that Drunix will eventually load.

The assembler turns textual assembly into an object file. An object file is a binary file containing machine code, data, symbols, and relocation information. Relocation information records places where the linker may need to adjust an address once all input objects are laid out together.

Rexc's i386 path is the current Drunix userland path. The generated assembly is assembled as a 32-bit x86 object file:

```
x86_64-elf-as --32 -o build/add.o build/add.s
```

The command is short, but the state change is significant. Before assembly, Rexc had text that described instructions. After assembly, the build has an ELF object file containing real i386 machine code.

Why the Runtime Is Part of the Program

A Rexc function named `main` is not the first instruction Drunix runs in a user process. Drunix starts at the executable entry point. That entry point belongs to the user runtime's startup object, commonly called `crt0.o`. The startup object receives the process stack prepared by the kernel, extracts `argc`, `argv`, and `envp`, calls `main`, and exits through the syscall path when `main` returns.

That startup path is why Rexc output must be linked with the Drunix runtime. The compiler can emit a perfectly good `main`, but it does not emit the whole process contract. The runtime supplies the bridge between the kernel's process entry state and the language-level function the program author wrote.

The runtime archive, `libc.a`, supplies the userland support code the program may call. An archive is a collection of object files. The linker pulls in only the archive members needed to satisfy unresolved symbols, which keeps the final program from dragging in the whole library when it only needs a small part.

The Linker Gives the Program Its Final Shape

The linker combines the startup object, the Rexc-generated object, and the runtime archive under the control of the Drunix user linker script. A linker script is a set of layout instructions for the linker. It decides where code and data live in the final address space and which symbol becomes the entry point.

For the current i386 Drunix path, the link has this shape:

Link input	Role
`crt0.o`	process entry and call into `main`
Rexc object	program code emitted from `.rx` source
`libc.a`	runtime and library support
`user.ld`	final executable layout

The order matters. The runtime archive should come after the program object so the linker sees the program's unresolved symbols first and can pull the needed archive members afterward.

ELF Is the Shared Contract

The final output is an **ELF** (Executable and Linkable Format) executable. ELF is the binary container format Drunix follows for user programs. It gives the kernel enough structure to find loadable segments, map them into a process, and start execution at the recorded entry point.

That is the quiet payoff of the whole pipeline. Drunix does not need a special “Rexc program” loader. Once the final link succeeds, the kernel sees an ELF binary with the same kind of process contract as a C program. The source language has disappeared from the operating system boundary.

The useful validation checks are simple: `file` should identify a 32-bit Intel 80386 executable, `readelf` should show an ELF32 executable for machine Intel 80386, and the first four bytes should be the ELF magic number.

Where the Compiler Is by the End of Chapter 6

Rexc can now participate in the Drunix userland story. A `.rx` source file can be parsed, checked, lowered, emitted as i386 assembly, assembled into an object file, and linked with Drunix’s startup object and runtime archive into a final ELF executable.

The operating system boundary remains clean. Drunix loads an ELF program. Rexc is responsible for producing code that fits the runtime contract. The next chapters can expand the language itself, but the first complete path from source text to Drunix user program is now visible end to end.